

LangChain — From Basics to Advanced

1. First: What is LangChain?

Let us start with one simple idea.

LangChain is a Python framework used to build applications powered by Large Language Models (LLMs).

For example, you can use an LLM such as Gemini or GPT directly:

```
User
↓
Question
↓
LLM
↓
Answer
```

This works well for simple questions.

But real-world AI applications need much more:

```
User
↓
Question
↓
Understand question
↓
Search documents
↓
Retrieve relevant information
↓
Send information to LLM
↓
Generate answer
↓
Use tools if required
↓
Remember previous conversation
↓
Final Answer
```

LangChain helps us build this complete pipeline.

2. Why do we need LangChain?

Suppose you want to build a **College AI Assistant**.

A student asks:

"What is the attendance requirement for the Machine Learning course?"

The information may be inside:

- College PDF
- Course website
- Database
- Google Drive
- College ERP

A simple LLM may not know your college's specific information.

So we need:

```
Student
↓
AI Assistant
↓
Search College Documents
↓
Find relevant information
↓
LLM
↓
Answer Student
```

LangChain provides components to build this type of system.

3. Simple Definition for Students

You can explain LangChain like this:

LangChain is a framework that connects an LLM with prompts, documents, databases, tools, memory, and other software to build intelligent AI applications.

Or even simpler:

LLM = Brain

LangChain = System that connects the brain with tools and information.

4. LangChain vs LLM

This is very important for beginners.

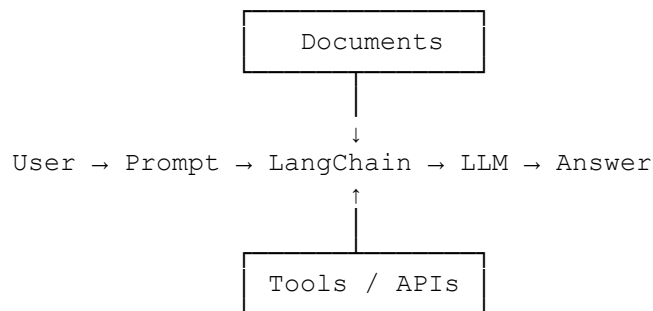
LLM

An LLM generates text.

```
Question
↓
LLM
↓
Answer
```

LangChain

LangChain helps us create a complete application around the LLM.



So:

LangChain does not replace the LLM.

It **connects and orchestrates** different components around the LLM.

5. Real-Life Analogy

Imagine a restaurant.

```
Customer
↓
Waiter
↓
Kitchen
↓
Chef
↓
Food
```

Here:

Restaurant	AI Application
Customer	User
Waiter	LangChain
Chef	LLM

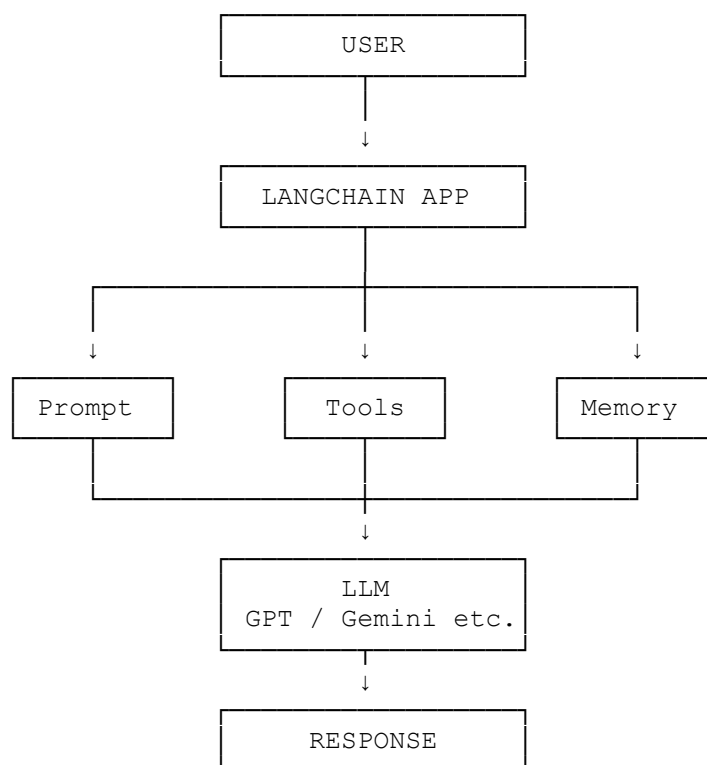
Restaurant	AI Application
Recipe/ingredients	Data
Kitchen tools	APIs/Tools
Food	Final answer

The **LLM is like the chef**.

LangChain is like the **system that coordinates the chef, ingredients, orders and tools**.

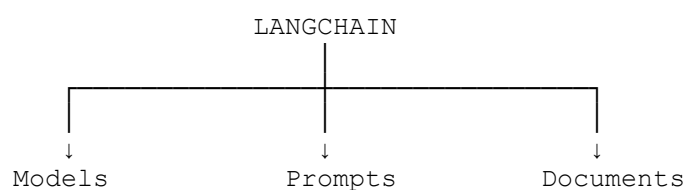
6. Basic LangChain Architecture

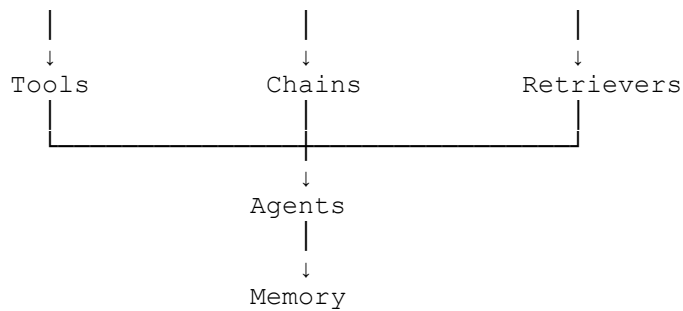
Here is a simple architecture you can show students:



7. Main Components of LangChain

We can divide LangChain into important building blocks.





Let's understand each one.

8. Model

The **model** is the AI brain.

Examples:

- OpenAI GPT models
- Google Gemini
- Anthropic Claude
- local models such as Llama

Example:

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash"
)
```

Conceptually:

```
Your Python Program
    ↓
LangChain
    ↓
Gemini
    ↓
Answer
```

9. Prompt

A prompt tells the LLM what to do.

For example:

Explain Machine Learning to a beginner.

A better prompt might be:

```
You are a Machine Learning teacher.
```

```
Explain Machine Learning in simple English.
```

```
Use:
```

1. Definition
2. Example
3. Advantages
4. Applications

```
Target audience: beginners.
```

LangChain provides tools for creating reusable prompts.

10. Prompt Template

Instead of writing a prompt every time, we create a template.

```
Explain {topic} to a beginner.
```

Then:

```
topic = Machine Learning
```

becomes:

```
Explain Machine Learning to a beginner.
```

If:

```
topic = Artificial Intelligence
```

then:

```
Explain Artificial Intelligence to a beginner.
```

This makes applications reusable.

11. Chains

This is one of the most important LangChain concepts.

A chain means connecting multiple steps together.

For example:

```
Question
↓
Prompt
↓
LLM
↓
Answer
```

This is a simple chain.

More advanced:

```
Question
↓
Generate Search Query
↓
Search Database
↓
Retrieve Information
↓
Send Context to LLM
↓
Generate Answer
```

This is also a chain.

Think:

Chain = Step 1 → Step 2 → Step 3 → Step 4

12. LCEL

Modern LangChain commonly uses **LCEL — LangChain Expression Language**.

You may see code like:

```
chain = prompt | llm
```

The `|` symbol means:

```
Prompt
↓
LLM
```

For example:

```
chain = prompt | llm | parser
```

means:

```
Prompt
```

↓
LLM
↓
Output Parser

Very easy way to remember:

LCEL connects components into a pipeline.

13. Output Parser

LLMs normally return text-like AI messages.

But sometimes our application needs structured output.

For example:

```
{  
  "name": "Rahul",  
  "age": 20,  
  "department": "Computer Engineering"  
}
```

An output parser helps convert model output into a useful format.

14. Documents

Real-world AI applications often need external information.

Examples:

PDF
Word
TXT
CSV
Website
Database

LangChain provides document-loading and processing capabilities.

For example:

College PDF
↓
Document Loader
↓
Text
↓

15. Document Loader

Suppose you have:

`Machine_Learning_Syllabus.pdf`

The application needs to read it.

Conceptually:

```
PDF
↓
Document Loader
↓
Text
↓
Document Objects
```

Different loaders can work with different sources.

16. Text Splitter

Suppose your PDF contains **100 pages**.

We should not necessarily send all 100 pages to the LLM every time.

So we divide the document into smaller pieces.

```
100 Page PDF
↓
Text Extraction
↓
Text Splitting
↓
```

Chunk 1
Chunk 2
Chunk 3
Chunk 4
...

These pieces are called **chunks**.

17. Embeddings

Now comes an important concept.

Suppose we have:

"Machine learning allows computers to learn from data."

We convert the meaning of this sentence into numbers.

These numbers are called an **embedding**.

Conceptually:

```
Text
↓
Embedding Model
↓
[0.12, -0.43, 0.78, 0.21, ...]
```

The numbers represent the **semantic meaning** of the text.

18. Vector Database

Where do we store these embeddings?

Usually in a **vector database**.

Examples include:

- FAISS
- Chroma
- Pinecone
- Weaviate
- Milvus

Architecture:

```
PDF
↓
Text
↓
Chunks
↓
Embeddings
↓
Vector Database
```

19. Retrieval

Now suppose a student asks:

"What is supervised learning?"

The system converts the question into an embedding.

Then searches the vector database for similar information.

```
Student Question
      ↓
Question Embedding
      ↓
Vector Search
      ↓
Relevant Chunks
```

20. RAG

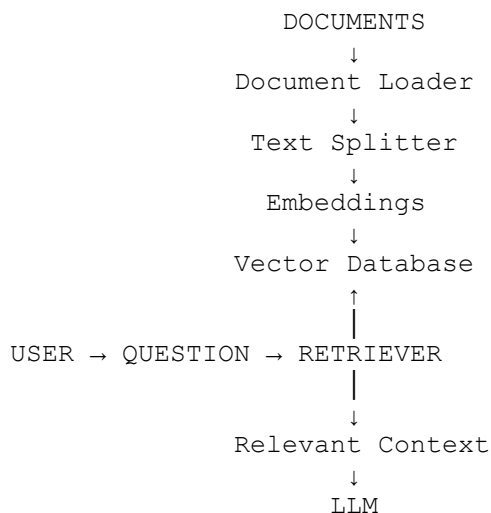
This leads us to one of the most important applications of LangChain:

RAG — Retrieval Augmented Generation

Very simple meaning:

First retrieve useful information, then ask the LLM to generate an answer using that information.

Flow:



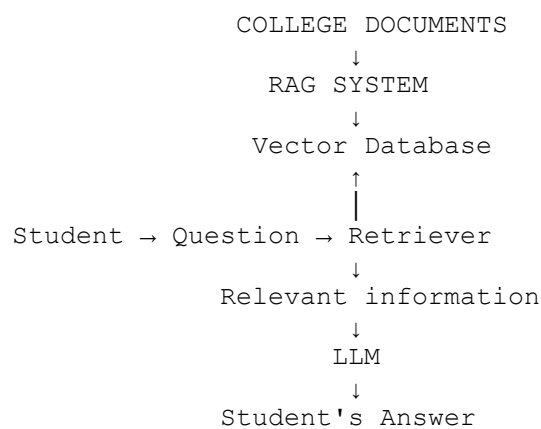
↓
ANSWER

21. Example: College RAG System

Imagine your college has:

Academic Regulations.pdf
Examination Rules.pdf
Attendance Rules.pdf
Course Syllabus.pdf

We create:



Student asks:

"How much attendance is required?"

The system retrieves the relevant college rule and generates the answer.

This is much more useful than relying only on the LLM's general knowledge.

22. Tools

Now LangChain becomes more powerful.

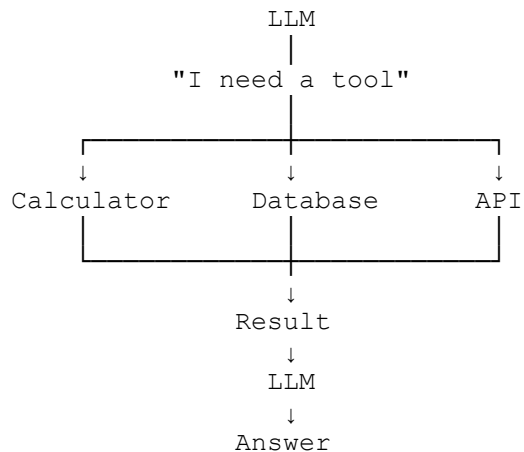
An LLM can be connected to external **tools**.

Examples:

Calculator
Weather API
Database
Search engine

Python
Email
CRM
ERP
Calendar

Architecture:



23. Why are Tools Important?

Suppose the user asks:

"What is 12567×876 ?"

The LLM can calculate approximately, but a calculator tool is more reliable.

Or:

"What is today's weather?"

The LLM's training data is not enough.

It needs a **weather API/tool**.

Therefore:

LLM
+
Tools
=
More capable AI application

24. Agents

This is where LangChain becomes more advanced.

An **agent** is an AI system that can decide **which tool to use and what steps to take**.

For example:

"Find the temperature in Pune and convert it from Celsius to Fahrenheit."

The agent may decide:

```
User Question
  ↓
  Agent
  ↓
Need weather information
  ↓
Weather Tool
  ↓
Temperature
  ↓
Need conversion
  ↓
Calculator Tool
  ↓
Final Answer
```

The important idea is:

A chain follows a predefined sequence.

An agent can decide the sequence dynamically.

25. Chain vs Agent

This distinction is extremely important for your students.

Chain

A → B → C → D

The sequence is predefined.

Agent

```
User → Agent {
               → Tool A
               → Tool B
               → Tool C
               → LLM
```

The agent decides what to do.

26. Memory / Conversation History

Imagine a chatbot.

User:

My name is Rahul.

AI:

Nice to meet you, Rahul.

Later:

What is my name?

If the application remembers:

Your name is Rahul.

Conversation history can be maintained and passed into the model.

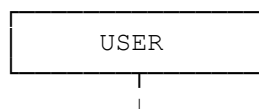
Modern LangChain applications often implement conversation state explicitly, rather than treating "memory" as a magical component.

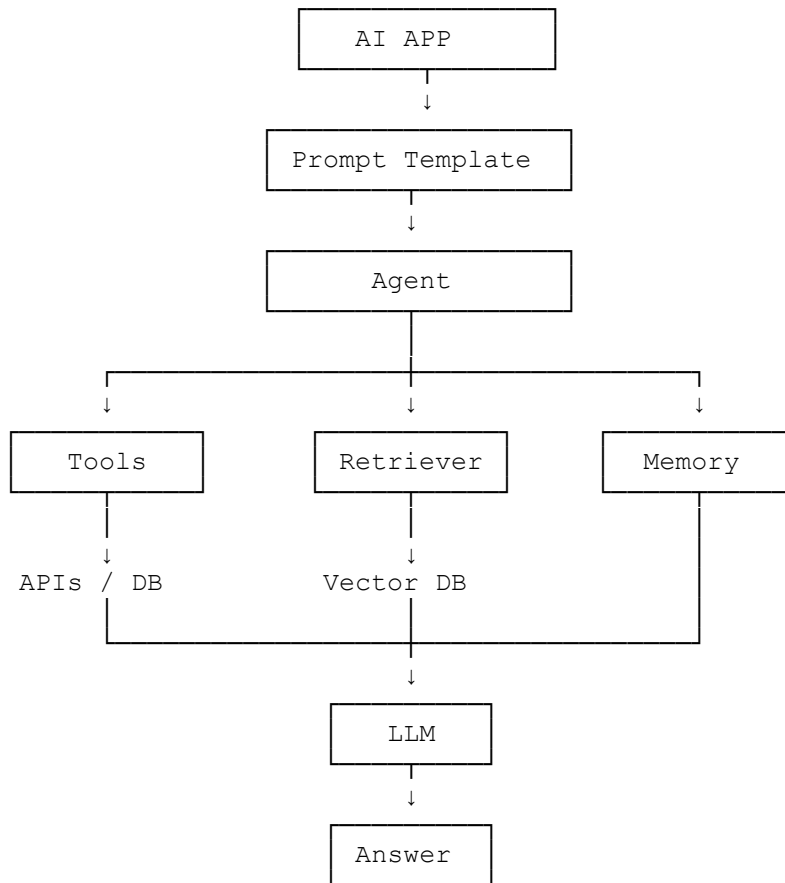
Conceptually:

```
graph TD
    A[User Message] --> B[Conversation State]
    B --> C[LLM]
    C --> D[Answer]
    D --> E[Updated State]
```

27. Advanced Architecture

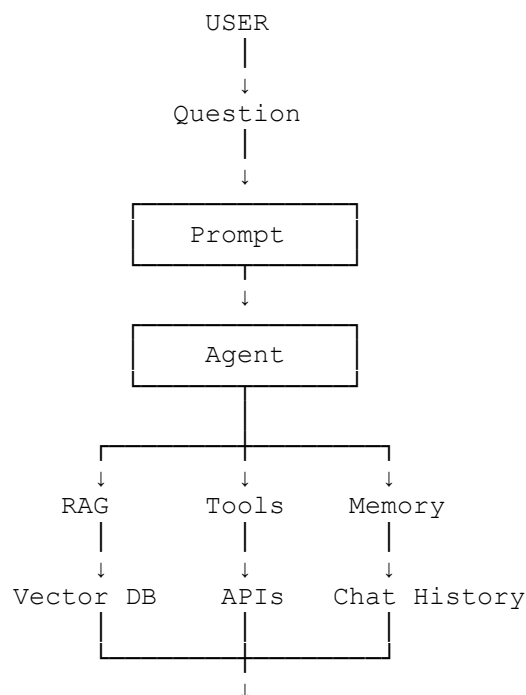
Now let's combine everything.





28. Complete LangChain Working

A practical LangChain application can work like this:



LLM
↓
Response

29. Industry Application 1 — Customer Support

Imagine an e-commerce company.

Customer asks:

"My order has not arrived. What should I do?"

The AI assistant can:

```
Customer
  ↓
AI Agent
  ↓
Check Order Database
  ↓
Find Order Status
  ↓
Check Company Policy
  ↓
Generate Response
  ↓
Customer
```

LangChain can connect:

```
LLM
+
Company Documents
+
Database
+
Order API
+
Agent
```

30. Industry Application 2 — Banking

Customer:

"What documents are required for a home loan?"

System:

Customer

↓
LangChain
↓
Retrieve Bank Policy
↓
LLM
↓
Answer

For account-specific questions:

Customer
↓
Agent
↓
Bank API
↓
Account Information
↓
LLM
↓
Response

31. Industry Application 3 — Healthcare

A hospital may build an internal assistant that searches:

Hospital Guidelines
Medical Protocols
Drug Information
Patient Documents

Flow:

Doctor
↓
Question
↓
Retriever
↓
Relevant Medical Documents
↓
LLM
↓
Answer / Summary

Important:

In healthcare, such systems should be designed as decision-support tools with appropriate clinical validation and safeguards, not as autonomous replacements for clinicians.

32. Industry Application 4 — Software Engineering

Developer asks:

"Explain this company's authentication module."

The system can search:

```
Git Repository
  ↓
Code Files
  ↓
Embeddings
  ↓
Vector Database
  ↓
Retriever
  ↓
LLM
  ↓
Explanation
```

This is useful for:

- Code explanation
 - Documentation generation
 - Bug investigation
 - Test generation
 - Developer copilots
-

33. Industry Application 5 — Education

This is particularly relevant to your teaching.

You could build:

AI Teaching Assistant

Documents:

```
Syllabus
  ↓
Lecture Notes
  ↓
Lab Manual
  ↓
```

Question Bank
↓
University Regulations

Student:

"Explain backpropagation in simple English."

System:

Student
↓
LangChain
↓
Retriever
↓
Course Material
↓
LLM
↓
Simple Explanation

It could also generate:

- MCQs
- Viva questions
- Lab guidance
- Summaries
- Personalized explanations
- Practice questions

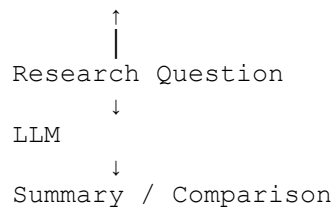
34. Industry Application 6 — Research Assistant

A researcher has:

100 Research Papers

The system can:

Research Papers
↓
Document Loader
↓
Chunking
↓
Embeddings
↓
Vector Database
↓
Retriever



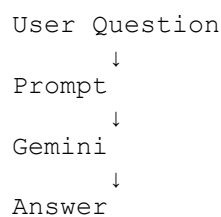
For example:

"Compare the optimization techniques used in these papers."

35. First Simple Python Program

Let's start with the **simplest possible LangChain application**.

Concept:



Install:

```
pip install langchain langchain-google-genai
```

Then Python:

```
# Import the Gemini chat model
from langchain_google_genai import ChatGoogleGenerativeAI

# Create the LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash"
)

# Ask a question
response = llm.invoke(
    "Explain Machine Learning to a beginner."
)

# Display the answer
print(response.content)
```

36. Explain This Program Step-by-Step

Step 1 — Import

```
from langchain_google_genai import ChatGoogleGenerativeAI
```

We import the LangChain integration that allows our Python program to communicate with Gemini.

Think:

```
Python
  ↓
LangChain
  ↓
Gemini
```

Step 2 — Create the model

```
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash"
)
```

Here:

```
llm
```

is our Python object representing the Gemini chat model.

We are basically saying:

"Create a connection to this Gemini model."

Step 3 — Send a question

```
response = llm.invoke(
    "Explain Machine Learning to a beginner."
)
```

`invoke()` means:

Send the input to the model and get a response.

Flow:

```
Python
  ↓
invoke()
  ↓
Gemini
  ↓
Response
```

Step 4 — Display answer

```
print(response.content)
```

The response contains information from the model.

`.content` gives us the actual text.

37. Now Introduce PromptTemplate

Let's make the program reusable.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate

# Create the Gemini model
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash"
)

# Create a reusable prompt template
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple English for beginners."
)

# Connect prompt and model
chain = prompt | llm

# Run the chain
response = chain.invoke({
    "topic": "Artificial Intelligence"
})

# Print the answer
print(response.content)
```

38. Understand the `|` Symbol

This line:

```
chain = prompt | llm
```

is very important.

It means:

```
Prompt
↓
```

LLM

Think of a pipeline:

```
Input
  ↓
Prompt Template
  ↓
Gemini
  ↓
Response
```

39. Add Output Parser

We can make the output cleaner.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Create the model
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash"
)

# Create prompt
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple English."
)

# Create output parser
parser = StrOutputParser()

# Create complete chain
chain = prompt | llm | parser

# Run the chain
answer = chain.invoke({
    "topic": "Neural Networks"
})

# Print final text
print(answer)
```

Now:

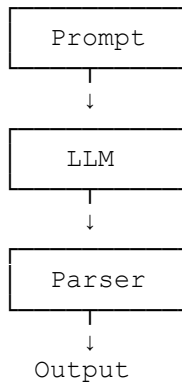
```
Input
  ↓
Prompt
  ↓
LLM
  ↓
Output Parser
  ↓
Clean Text
```

40. This is a LangChain Chain

The following:

```
chain = prompt | llm | parser
```

is essentially:



This is the fundamental idea behind composable LangChain applications.

41. A More Practical Program

Let's build a small **AI Teacher**.

The user gives a topic.

The system generates:

1. Definition
2. Simple explanation
3. Example
4. Applications

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# -----
# STEP 1: Create the Gemini LLM
# -----

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash"
)

# -----
# STEP 2: Create the prompt template
```

```

# -----

prompt = ChatPromptTemplate.from_template("""
You are an expert teacher.

Explain the following topic to engineering students:

Topic: {topic}

Use the following structure:

1. Definition
2. Simple Explanation
3. Real-Life Example
4. Industry Applications
5. Advantages
6. One simple example
""")

# -----
# STEP 3: Create output parser
# -----

parser = StrOutputParser()

# -----
# STEP 4: Create LangChain pipeline
# -----

chain = prompt | llm | parser

# -----
# STEP 5: Give input
# -----

topic = "Machine Learning"

# -----
# STEP 6: Run the chain
# -----

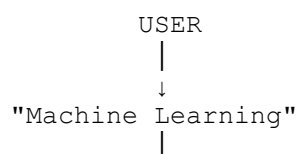
answer = chain.invoke({
    "topic": topic
})

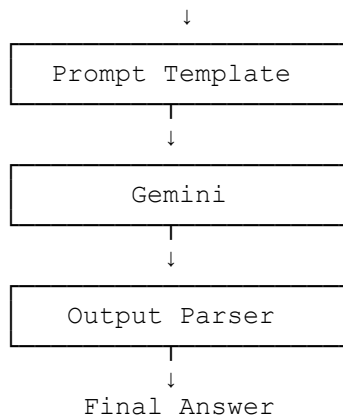
# -----
# STEP 7: Display result
# -----

print(answer)

```

42. Complete Working of This Program





43. RAG — Advanced LangChain

Now we move to a much more important application.

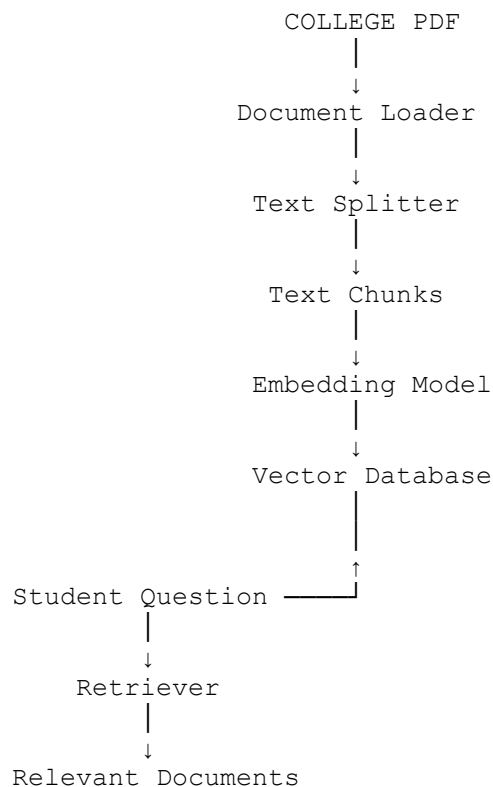
Suppose:

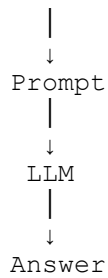
`College_Manual.pdf`

contains 100 pages.

We want students to ask questions about it.

Complete architecture:





44. RAG Python Structure

A simplified RAG program conceptually looks like:

```
# 1. Load document
documents = loader.load()

# 2. Split document
chunks = splitter.split_documents(documents)

# 3. Create embeddings
embeddings = ...

# 4. Store embeddings
vector_store = ...

# 5. Create retriever
retriever = vector_store.as_retriever()

# 6. Retrieve relevant documents
docs = retriever.invoke(question)

# 7. Send retrieved context to LLM
answer = llm.invoke(...)

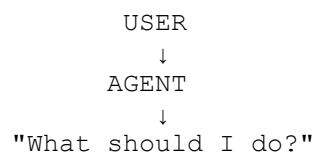
# 8. Display answer
print(answer)
```

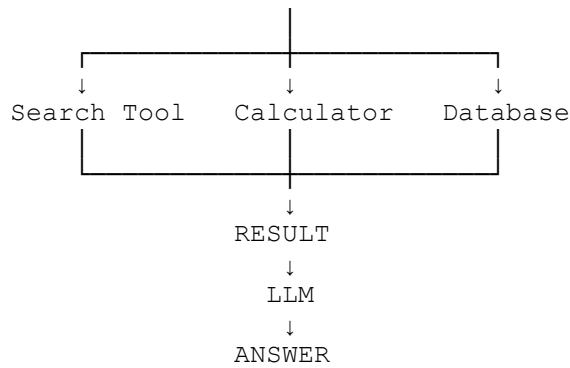
The key idea is:

The LLM answers using retrieved information instead of depending only on its training knowledge.

45. Agent Architecture

Now let's move to another advanced concept.





The agent can select tools according to the task.

46. LangChain and Agentic AI

This is especially important for your **Agentic AI & LLM System** course.

You can think of the evolution as:

Level 1
Simple LLM
↓
Question → Answer

Level 2
Prompt Engineering
↓
Better Question → Better Answer

Level 3
LangChain
↓
Prompt + LLM + Components

Level 4
RAG
↓
LLM + External Knowledge

Level 5
Tools
↓
LLM + External Actions

Level 6
Agents
↓
LLM + Reasoning + Tools + Dynamic Decisions

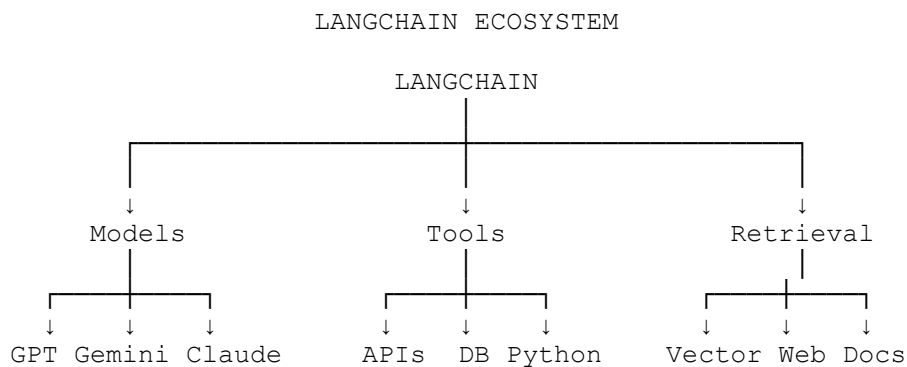
Level 7
Agentic AI System

↓
Multiple agents
+ Tools
+ Memory
+ RAG
+ Planning
+ Execution
+ Monitoring

47. LangChain Ecosystem

Modern LangChain applications can involve several pieces.

A useful conceptual ecosystem is:



You may also encounter **LangGraph**, which is used for more complex, stateful agent workflows.

48. LangChain vs LangGraph

For beginners:

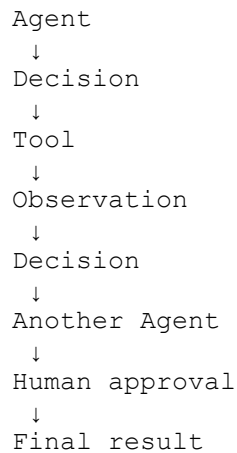
LangChain

Good for:

LLM
Prompt
RAG
Tools
Chains
Agents
Integrations

LangGraph

Useful when workflows become more complex:



Think:

LangChain = building blocks

LangGraph = controlling complex workflows and agent state

49. Important LangChain Features

You can summarize the features as:

1. LLM Integration

Connect different LLM providers.

2. Prompt Management

Create reusable prompts.

3. Chains

Connect multiple AI operations.

4. RAG

Connect LLMs with external knowledge.

5. Document Processing

Load and process documents.

6. Embeddings

Convert text into vectors.

7. Vector Stores

Store and search semantic information.

8. Tools

Connect external APIs and software.

9. Agents

Allow LLMs to select tools and actions.

10. Structured Output

Convert responses into structured data.

11. Streaming

Show generated responses progressively.

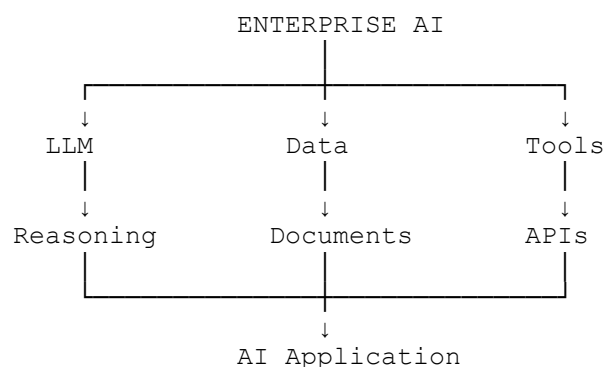
12. Observability

Monitor and debug complex LLM applications using the broader LangChain ecosystem.

50. Why Industry Uses LangChain

Because an enterprise AI application usually needs more than an LLM.

For example:



LangChain helps developers connect these components.

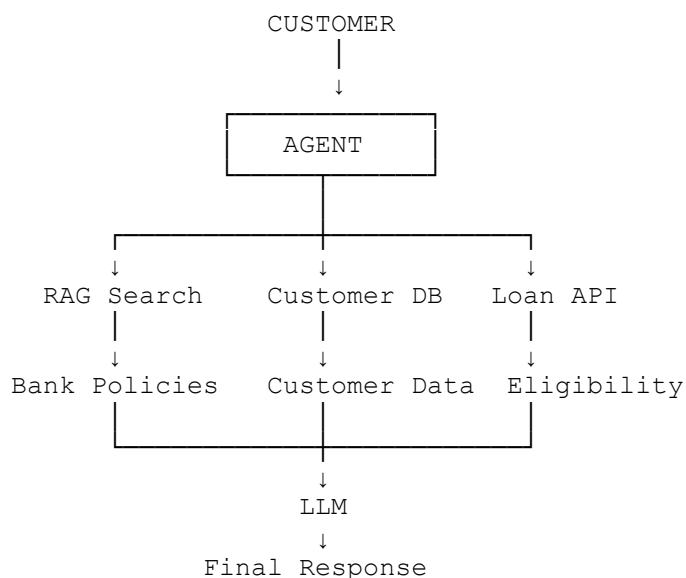
51. One Complete Industry Example

Let's design an **AI Banking Assistant**.

Customer asks:

"Can I get a home loan, and what documents do I need?"

The system could work like this:



This is much closer to a real enterprise GenAI system than:

Question → ChatGPT

52. Important Security Consideration

When building industry applications, LangChain itself does **not automatically make an application secure**.

Developers must consider:

- Authentication
- Authorization
- Data privacy
- Prompt injection
- Tool permissions
- Sensitive information
- API security

- Logging
- Human approval for high-risk actions
- Hallucination
- Data leakage

For example:

```
User
↓
Agent
↓
Bank Database
```

You should **not** allow an unrestricted agent to access every database operation.

Use:

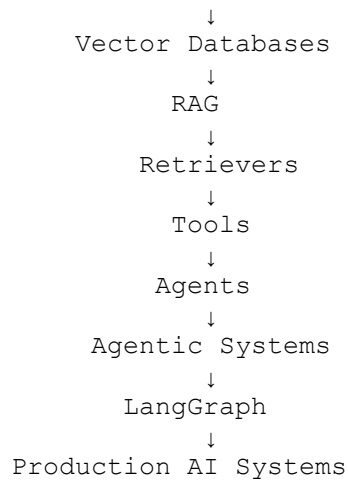
```
Agent
↓
Permission Check
↓
Allowed Tool
↓
Limited Data
```

53. Beginner → Advanced Learning Roadmap

For your students, I would teach LangChain in this order:

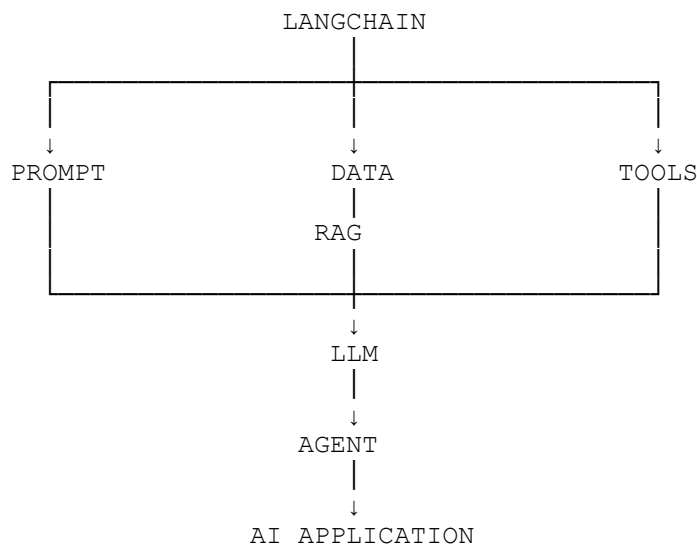
```
LANGCHAIN LEARNING PATH

START
↓
What is LLM?
↓
What is LangChain?
↓
LLM Integration
↓
Prompt Templates
↓
Chains
↓
LCEL
↓
Output Parsers
↓
Document Loaders
↓
Text Splitters
↓
Embeddings
```



54. The Most Important Mental Model

If you remember only one diagram, remember this:



55. Simple One-Line Explanation of Each Concept

This table is excellent for classroom teaching:

Concept	Simple Meaning
LLM	AI brain that understands and generates language
LangChain	Framework for building applications around LLMs
Prompt	Instructions given to the LLM
Prompt Template	Reusable prompt

Concept	Simple Meaning
Chain	Sequence of connected steps
LCEL	Language used to connect LangChain components
Document Loader	Reads external documents
Text Splitter	Divides large text into smaller chunks
Embedding	Represents meaning as numbers
Vector Database	Stores and searches embeddings
Retriever	Finds relevant information
RAG	Retrieve information + Generate answer
Tool	External capability such as API/database/calculator
Agent	AI that decides which tools/actions to use
Memory/State	Maintains useful conversation/workflow information
LangGraph	Framework for complex stateful agent workflows

56. Final Big Picture

A traditional program may look like:

```

Input
↓
Rules
↓
Output

```

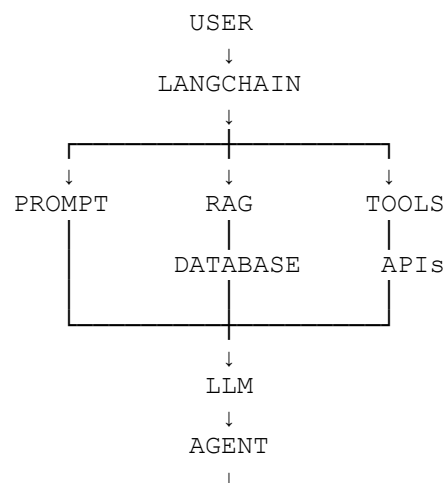
A simple LLM application:

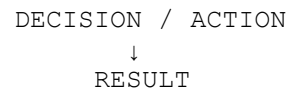
```

Input
↓
LLM
↓
Output

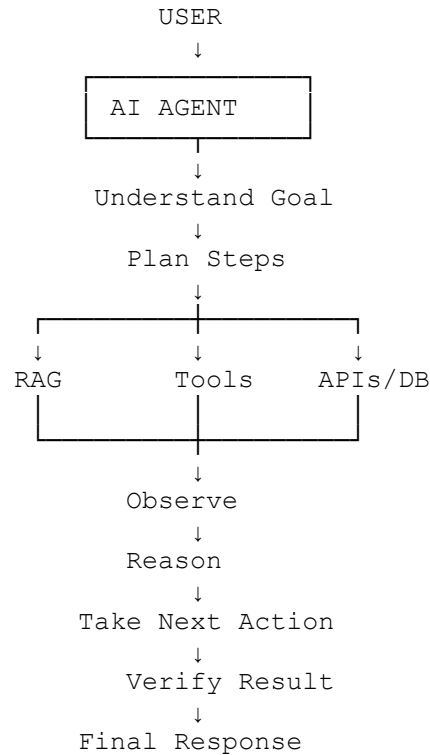
```

A LangChain application:





And a sophisticated **Agentic AI system**:



The simplest sentence to teach students

LangChain provides building blocks for connecting LLMs with prompts, data, retrieval systems, tools, and workflows so that we can build real-world Generative AI and Agentic AI applications.

And the progression to remember is:

LLM → Prompt → Chain → RAG → Tools → Agents → Agentic AI.